Contents

Sorting Techniques & Cyclic Sort — Complete Interview Handbook

Pattern #7 of 28 | DSA Pattern Mastery Series Audience: Senior/Staff SWE interviews (Google, Amazon, Meta, Microsoft, Uber, Airbnb, Atlassian, Grab, Careem, Noon, Talabat, Dubai/Saudi & India Tier-1/Tier-2 product companies) **Companion:** Pairs with Striver's A2Z DSA Course — Sorting section

---

Table of Contents

SECTION 1 — Pattern Overview

1.1 What Is This Pattern?

This pattern covers both (a) **general sorting techniques** (comparison-based: merge sort, quicksort; non-comparison: counting sort, radix sort/bucket sort) as foundational tools, and (b) **Cyclic Sort**, a specialized O(n) in-place technique for arrays containing numbers in a known, bounded range (typically `1..n` or `0..n-1`), where each value's own magnitude tells you exactly which index it belongs at.

1.2 Why Was This Pattern Invented?

Comparison-based sorts are bounded by the information-theoretic Ω(n log n) lower bound because they only extract "less than/greater than" information per comparison. But when the input has extra structure — like "these n numbers are exactly a permutation of 1..n, possibly with duplicates/missing values" — you don't need general-purpose comparison sorting at all: **you can place each element directly at its "home" index in one pass**, because the value itself encodes the destination. This insight, called Cyclic Sort, achieves O(n) time using the constraint that comparison sorting can't exploit.

1.3 Real Intuition Behind The Pattern

Imagine a hotel with rooms numbered 1 to n, and n guests each wearing a badge with their assigned room number. Instead of comparing guests to each other to figure out an order, you can just **directly walk each guest to the room matching their badge number** — no comparisons needed, just direct placement. That's Cyclic Sort.

1.4 Mental Model

For an array where `arr[i]` "should" be at index `arr[i] - 1` (1-indexed values, 0-indexed array), repeatedly swap the element at your current position into its correct home until the element that lands in your current position is already correctly placed (or you find a duplicate/mismatch), then move on.

1.5 Visual Explanation

```
arr = [3, 1, 5, 4, 2]   (values 1..5, should end sorted [1,2,3,4,5])

i=0: arr[0]=3, home index for 3 is index 2. arr[2]=5  3, so swap:
     [5, 1, 3, 4, 2] → still arr[0]=5, home index 4. arr[4]=2  5, swap:
     [2, 1, 3, 4, 5] → arr[0]=2, home index 1. arr[1]=1  2, swap:
     [1, 2, 3, 4, 5] → arr[0]=1, home index 0. Correct! Move i forward.
i=1..4: already correctly placed. Done. Total swaps: 3, one pass overall O(n).
```

1.6 Simple Analogy

Cyclic Sort is like a **library reshelving system where every book's call number IS its shelf position** — instead of comparing books to each other to determine order, you just read each book's number and put it directly on the matching shelf, swapping out whatever was there and repeating for that book.

1.7 When Should I Immediately Think About Using This Pattern?

- Array contains **n numbers in range [1, n]** (or **[0, n-1]**), possibly with **duplicates or missing values**.
- Problem asks to **find missing/duplicate numbers** in O(n) time, O(1) space.

- Problem says “array contains numbers from 1 to n” explicitly.
- More generally (non-cyclic sort): when you need data **sorted before applying Two Pointers or Greedy**, or when a **specific sorting algorithm’s properties** (stability, in-place, O(n) counting sort for small ranges) matter to the solution.

SECTION 2 — Recognition Guide

2.1 Keywords

Keyword	Signal
“array contains n numbers from 1 to n”	Direct Cyclic Sort signal
“find the missing number”	Cyclic Sort or sum-formula trick
“find the duplicate number”	Cyclic Sort or Fast & Slow Pointers (functional graph)
“in-place,” “O(1) extra space,” bounded range	Cyclic Sort signal
“sort colors,” “sort 0s 1s 2s”	Dutch National Flag (bounded-range partition sort)
“smallest missing positive integer”	Cyclic Sort adjacent variant

2.2 Hidden Hints

Constraint like `nums.length == n` and `1 <= nums[i] <= n` (or `0 <= nums[i] <= n`) is the single strongest, almost unambiguous, signal for Cyclic Sort — this exact constraint phrase appears verbatim across dozens of LeetCode problems.

2.3 Interview Clues

Interviewer emphasizes “the array has exactly n elements, with values bounded by n” — this bounded-range framing is a deliberate setup for Cyclic Sort, distinguishing it from general “sort this array” requests.

2.4 Common Trick Words

“Missing,” “duplicate,” “first missing positive,” “disappeared” — these are the classic Cyclic Sort family vocabulary.

2.5 What Interviewers Expect

Recognition that this achieves O(n) time and O(1) space — better than sorting (O(n log n)) or hashing (O(n) space) — and correct handling of the swap loop to avoid infinite loops when duplicates are present.

2.6 When NOT To Use This Pattern

- Values are **not bounded to a range matching the array’s length** — Cyclic Sort’s core trick (value = index) doesn’t apply.
- You need a **general-purpose, comparison-based sort** for arbitrary data — use standard library sort (Merge Sort/Quicksort/Timsort), don’t try to force Cyclic Sort.
- Stability matters (preserving relative order of equal elements) — Cyclic Sort doesn’t guarantee stability; use a stable sort (Merge Sort, or a stable counting sort variant) instead.

SECTION 3 — Decision Framework

```
Does the array contain exactly n values within a KNOWN BOUNDED range
matching indices (e.g., 1..n or 0..n-1)?

       Yes → USE CYCLIC SORT (O(n) time, O(1) space, in-place placement)

        No

Is the value range small and DISCRETE (e.g., 0,1,2 only, or bounded integers)?

       Yes → USE COUNTING SORT / DUTCH NATIONAL FLAG (O(n) time, O(k) or O(1) space)

        No

Do you just need GENERAL sorted order with no special structure to exploit?

       Yes → USE COMPARISON SORT (Merge Sort O(n log n) stable, Quicksort O(n log n) avg in-place)

        No

Do you need the k SMALLEST/LARGEST elements, not a full sort?

       Yes → USE HEAP / QUICKSELECT instead (Pattern #17) - avoid sorting the entire array unnecessarily
```

Why: Cyclic Sort exploits a very specific structural constraint (value range == index range) that comparison sorts cannot use — when that constraint holds, it strictly dominates general sorting in both time and space. When it doesn't hold, forcing Cyclic Sort is either impossible or incorrect.

SECTION 4 — Why This Pattern Works

Mathematical: If array `arr` has length `n` and contains values from `1` to `n` (allowing duplicates/missing), then a **correctly sorted** version has `arr[i] = i + 1` for every `i`. Cyclic Sort directly enforces this invariant by placing each encountered value at its target index via swapping — each swap places **at least one** element into its final correct position (the element being swapped *into* the current position may or may not be correct yet, but the element swapped *out* is now guaranteed correctly placed if it matches its new home... more precisely: each swap operation places the value being moved *out of the current slot* into a slot where it may or may not immediately be correct, but the crucial invariant is that every swap strictly increases the count of "index i with arr[i] == i+1"-satisfying positions in aggregate over the whole process, since each element is only swapped when it's not yet home, and once placed home it is never moved again).

Logical: Since each of the `n` positions receives its final correct value in at most one "settling" swap on average, and the total number of swaps across the entire process is bounded by `n` (each swap correctly places at least one element permanently), the total work is O(n), not $O(n^2)$ despite the nested-looking `while` loop inside a `for` loop.

Intuitive: Every number "knows" exactly where it belongs (its own value tells you the index) — so instead of comparing numbers to discover order, you directly teleport each number home.

Correctness Proof: *Invariant:* after processing index `i` (fully, including all its internal swaps), positions `0..i` all satisfy `arr[j] == j+1` (or are recognized as a duplicate/missing marker). *Base case:* trivially true before processing begins. *Inductive step:* the inner while-loop at position `i` only terminates when `arr[i] == i+1` or the value at `i` is a duplicate of the value already at its correct home — either way, position `i` is settled correctly (or correctly identified as anomalous) without disturbing already-settled positions `0..i-1` (since a swap only ever moves a value to *its own* home

index, which by the invariant is always `i` for unsettled values). *Termination:* after processing all `n` positions, the array satisfies the sorted invariant or all anomalies (missing/duplicate) are identifiable via a final linear scan. **QED.**

SECTION 5 — Algorithm Framework

5.1 Step-by-Step Framework (Cyclic Sort)

1. Initialize `i = 0`.
2. While `i < n`: compute `correctIndex = arr[i] - 1` (for 1-indexed values).
3. If `arr[i] != arr[correctIndex]`, swap `arr[i]` and `arr[correctIndex]`.
4. Else (already correct, or duplicate found), increment `i`.
5. After the loop, do a final pass: any index `i` where `arr[i] != i + 1` reveals a missing (`i+1` is missing) or duplicate (`arr[i]` appears more than once) value.

5.2 General Template

```
function cyclicSort(arr):
    i = 0
    n = length(arr)
    while i < n:
        correctIndex = arr[i] - 1
        if arr[i] != arr[correctIndex]:
            swap(arr[i], arr[correctIndex])
        else:
            i = i + 1
    return arr
```

5.3 Finding Missing Number (Template)

```
function findMissing(arr):
    cyclicSort(arr)   # places each value at its home index where possible
    for i in range(0, length(arr)):
        if arr[i] != i + 1:
            return i + 1
    return length(arr) + 1   # all present, missing is n+1
```

5.4 Interview Thinking Process

1. "The array has n elements with values bounded by n — this is a strong signal for Cyclic Sort."
2. "I'll place each element at its home index (value - 1) via in-place swapping, achieving O(n) time and O(1) space — beating a full sort's O(n log n)."
3. "I need to be careful about the swap-vs-advance condition to avoid an infinite loop when a duplicate is encountered (if `arr[i] == arr[correctIndex]` already, I must advance `i`, not attempt another swap)."
4. "After placement, a final linear scan reveals any missing/duplicate values by checking `arr[i] != i + 1`."

SECTION 6 — Time & Space Complexity

Case	Time	Space	Why
Worst Case	O(n)	O(1)	Each element is swapped at most once into its final correct position; total swaps bounded by n
Average Case	O(n)	O(1)	Same bound regardless of initial arrangement
Best Case	O(n) (already sorted still requires one pass to verify)	O(1)	Must still scan once
Amortized	O(n) despite the nested while-loop appearance	O(1)	Each swap strictly makes progress (places one element home), so total swaps ≤ n across the entire run

Comparison sorts (for context): Merge Sort O(n log n) time, O(n) space (stable); Quicksort O(n log n) average / O(n^2) worst, O(log n) space (in-place, unstable); Counting Sort O(n + k) time, O(k) space (k = range size, stable if implemented carefully).

SECTION 7 — Edge Cases

Edge Case	Example	Handling
Empty array	`[]`	Return immediately, no missing/duplicate to find
Single element, correct	`[1]`	Already home, loop terminates immediately
Single element, "wrong" value out of range	`[2]` in a size-1 array expecting `[1]`	Value out of valid index range — must guard against negative/out-of-bounds correctIndex
All duplicates	`[1,1,1]`	Swap condition correctly detects `arr[i] == arr[correctIndex]` and advances without infinite looping
All values missing except one	`[1]` in a size-5 array (rest are placeholders/zero)	Final scan correctly identifies all missing values
Value out of the expected range entirely	`[7]` in a 1..n array with n=5	Must explicitly check `1 <= arr[i] <= n` before attempting to compute correctIndex, else skip/guard

Edge Case	Example	Handling
Negative numbers mixed in (variant problems)	`[-1, 4, 3, 1]`	Common in "First Missing Positive" — must explicitly skip/guard negative and zero values, only cyclic-sort valid positive range values

Common mistakes: forgetting to guard against out-of-range values before computing `correctIndex`, causing an array-index-out-of-bounds error; infinite loops when the swap condition doesn't correctly detect "already correct or duplicate" and advance `i`.

SECTION 8 — Pros & Cons

Advantages: O(n) time, O(1) space — strictly better than both general sorting (O(n log n)) and hashing (O(n) space) for its narrow, specific use case. **Disadvantages:** Only applies when values are bounded within a range matching the array's indices; doesn't generalize to arbitrary-range or arbitrary-type data. **Trade-offs:** Cyclic Sort (O(n), O(1), narrow applicability) vs. Hashing (O(n) time, O(n) space, broadly applicable) vs. general Sort (O(n log n), O(1) or O(n) space, broadly applicable but slower). **Limitations:** Not stable; not applicable to floating-point, string, or unbounded-range data; requires the array to be mutable in-place. **Inefficient when:** the range doesn't match the array size (then it's not really "cyclic sort" — you'd need a different indexing scheme or a hashmap instead).

SECTION 9 — Real World Applications

Domain	Application
Amazon	Detecting missing/duplicate SKU IDs in a bounded ID range during warehouse inventory reconciliation
Google	Compact bitset/ID-slot allocation systems where IDs are pre-allocated in a known dense range
Meta	Detecting gaps in sequentially-assigned internal IDs (e.g., shard IDs, partition IDs) in bounded ranges
Databases	Auto-increment ID gap detection in a known bounded range during data integrity checks
Operating Systems	Process ID (PID) allocation and reuse — detecting free PID slots in a bounded range efficiently
Networking	Detecting missing/duplicate sequence numbers in a bounded window of packet IDs
Distributed Systems	Detecting missing shard/partition assignments in a bounded partition-ID space during rebalancing
General Sorting (Merge/Quick/Counting Sort)	Ubiquitous — database query result ordering, search ranking, log timestamp ordering, virtually every "ORDER BY" operation

---

SECTION 10 — Interview Strategy

How seniors answer: They immediately notice the "values from 1 to n" constraint and explicitly contrast Cyclic Sort's O(n)/O(1) against a hashmap's O(n)/O(n) and a full sort's O(n log n)/O(1), justifying the choice before coding.

How juniors answer: They often default to sorting the array first (O(n log n)) or using a hash set (O(n) space) without recognizing the tighter O(n) time / O(1) space solution the bounded-range constraint enables.

Typical follow-ups: "Can you do this without extra space?" (pushes toward Cyclic Sort if a hashmap was proposed first). "What if there are multiple missing numbers?" "What if the array can contain negative numbers or zero?" (First Missing Positive variant — requires explicit range guarding).

Optimization questions: "Can you find all duplicates in one pass without extra space?" (Yes — Cyclic Sort naturally reveals all anomalies in the final verification scan).

SECTION 11 — Pattern Variations

Variation	Description	Example
Basic Cyclic Sort	Place each value at value-1 index	Missing Number
Find All Missing Numbers	Final scan reveals all mismatches	Find All Numbers Disappeared in an Array
Find All Duplicates	Final scan reveals all duplicate values	Find All Duplicates in an Array
Find the Single Duplicate	Constrained to exactly one duplicate	Find the Duplicate Number (also solvable via Fast/Slow Pointers)
First Missing Positive (with negative/out-of-range guarding)	Skip invalid values during placement	First Missing Positive
Dutch National Flag (bounded discrete values)	Three-way partition for exactly {0,1,2}	Sort Colors
Counting Sort	Non-comparison sort for small integer ranges	Sort an array of ages/scores in a known small range

---

SECTION 12 — Comparison With Other Patterns

Pattern	Difference	When To Prefer
Hashing	O(n) space but works for any value range, no bounded-index constraint needed	Values aren't bounded to match array indices
Fast & Slow Pointers	Also solves "Find the Duplicate Number" via functional-graph cycle detection, without modifying the array	When you cannot mutate the input array (Cyclic Sort requires in-place swaps)

Pattern	Difference	When To Prefer
Comparison Sort (Merge/Quick)	General-purpose, no special structure required, but O(n log n)	Arbitrary unbounded data, need actual full sorted order
Counting Sort	Exploits small discrete range, not necessarily matching array length	Range is small but doesn't align 1:1 with array indices

Comparison Table

Aspect	Cyclic Sort	Hashing	Merge Sort
Time	O(n)	O(n)	O(n log n)
Space	O(1)	O(n)	O(n)
Requires bounded range = array size	Yes	No	No
Mutates input	Yes	No	Depends on implementation
Stable	No	N/A	Yes

SECTION 13 — Problem Classification

Difficulty	Characteristics	Examples
Easy	Direct single missing/duplicate detection	Missing Number, Find the Duplicate Number (basic framing)
Medium	Multiple missing/duplicate detection, guarded ranges	Find All Numbers Disappeared in an Array, Find All Duplicates in an Array
Hard	Out-of-range guarding with negative numbers	First Missing Positive
Very Hard	Combined with additional constraints (e.g., set matrix operations, multi-array reconciliation)	Set Mismatch variants at scale, custom multi-constraint cyclic sort problems

---

SECTION 14 — 30 Interview Problems

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
1	Missing Number	Easy	Amazon, Microsoft, Meta	Values 0..n, one missing	Foundational cyclic sort
2	Find All Numbers Disappeared in an Array	Easy	Amazon, Meta	Values 1..n, multiple missing	Multiple-anomaly detection

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
3	Find All Duplicates in an Array	Medium	Amazon, Meta, Microsoft	Values 1..n, each 1-2 times	Duplicate detection via placement
4	Find the Duplicate Number	Medium	Amazon, Meta, Google, Microsoft	Single duplicate in 1..n range	Cyclic Sort vs Fast/Slow contrast
5	First Missing Positive	Hard	Amazon, Meta, Microsoft, Google	Guarded range placement with negatives	Advanced range guarding
6	Set Mismatch	Easy	Amazon	One missing + one duplicate simultaneously	Dual-anomaly detection
7	Sort Colors	Medium	Amazon, Meta, Microsoft, Google	Dutch National Flag (bounded discrete sort)	Three-way partition sort
8	Kth Missing Positive Number	Easy	Amazon	Related counting logic (often binary search instead)	Contrast with binary search
9	Couples Holding Hands	Hard	Google, Amazon	Related in-place swapping logic	Advanced in-place swap reasoning
10	Sort Array By Parity	Easy	Amazon	Partition-based sort	Predicate-based in-place partition
11	Wiggle Sort	Medium	Google, Amazon	In-place rearrangement via sorting logic	Rearrangement mastery
12	Wiggle Sort II	Medium	Google	Advanced in-place rearrangement	Median + rearrangement combination
13	Merge Sorted Array	Easy	Amazon, Meta, Microsoft	Related merge-based sort (contrast)	Merge mechanics
14	Sort an Array (general sort implementation)	Medium	Amazon, Google, Microsoft	Implement Merge Sort / Quicksort from scratch	Foundational sorting algorithm mastery
15	Largest Number	Medium	Amazon, Google	Custom comparator sorting	Comparator design

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
16	Relative Sort Array	Easy	Amazon, Google	Counting sort with custom order	Counting sort variant
17	H-Index	Medium	Google, Amazon	Sorting + threshold counting (or cyclic-sort-style bucket)	Sort-based threshold problems
18	Maximum Gap	Hard	Google, Amazon	Bucket sort / radix sort application	Non-comparison sort application
19	Sort List (linked list merge sort)	Medium	Microsoft, Amazon	Merge sort applied to linked lists	Cross-pattern (Linked List + Sort)
20	Meeting Rooms	Easy	Amazon, Meta	Sort-based interval overlap detection	Sorting as pre-processing step
21	Meeting Rooms II	Medium	Amazon, Meta, Google	Sort + min-heap combination	Sort + heap hybrid
22	Non-overlapping Intervals	Medium	Amazon, Google	Sort by end time + greedy	Sort + greedy combination
23	Valid Anagram	Easy	Amazon, Meta	Sorting or counting-based comparison	Basic counting sort application
24	Top K Frequent Elements	Medium	Amazon, Meta, Google	Bucket sort by frequency	Frequency-based bucket sort
25	Custom Sort String	Medium	Amazon, Google	Counting sort with custom priority order	Custom-order counting sort
26	Pancake Sorting	Medium	Google	Specialized in-place sort via flip operations	Constrained sorting operations
27	Sort Array by Increasing Frequency	Easy	Amazon	Counting + custom comparator sort	Frequency-based custom sort
28	Minimum Number of Swaps to Make the String Balanced (contrast)	Medium	Amazon	Contrast: greedy, not sort-based	Pattern-boundary awareness

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
29	Array With Elements Not Equal to Average of Neighbors (contrast)	Medium	Google	Contrast: re-arrangement without cyclic sort's bounded-range assumption	Recognizing pattern limits
30	Minimum Swaps to Sort an Array (custom/interview variant)	Medium	Amazon, Microsoft (conceptually asked)	Cyclic Sort used to compute minimum swaps via cycle counting	Advanced cycle-counting application

---

SECTION 15 — Common Mistakes

1. Forgetting to guard against out-of-range values before computing `correctIndex = arr[i] - 1`, causing index-out-of-bounds errors. *Fix:* explicitly check `1 <= arr[i] <= n` before attempting placement.
2. Infinite loops from an incorrect swap-vs-advance condition — always compare `arr[i]` to `arr[correctIndex]`, not just check if `i == correctIndex`. *Fix:* use the exact condition `if arr[i] != arr[arr[i]-1]: swap; else: i++`.
3. Applying Cyclic Sort to data where the value range doesn't match the array size — this silently produces wrong results since the "value = index" assumption doesn't hold. *Fix:* always verify the bounded-range constraint explicitly.
4. Forgetting the final verification scan after placement — the placement loop alone doesn't directly answer "which number is missing/duplicated"; a separate O(n) scan comparing `arr[i]` to `i+1` is required. *Fix:* always follow placement with an explicit verification pass.
5. Using Cyclic Sort when stability or non-mutation of the input is required. *Fix:* recognize these constraints upfront and use hashing instead if either applies.

Why people fail: the swap logic looks deceptively simple, but the exact condition for "when to swap vs. when to advance" is easy to get subtly wrong, and the resulting bug (infinite loop or incorrect answer) can be hard to spot without a careful dry run — candidates who don't trace through a duplicate-containing example on paper often miss this.

SECTION 16 — Optimization Techniques

- **Time:** Already optimal at O(n) for its narrow use case — no further asymptotic improvement possible.
- **Space:** Already O(1) — the entire point versus hashing.
- **Readability:** Extract the "compute correct index" and "swap" logic into clearly commented lines; explicitly comment the loop invariant ("i only advances when position i is settled").
- **Interview performance:** Explicitly state the O(n)/O(1) vs O(n)/O(n) (hashing) vs O(n log n)/O(1) (sorting) trade-off comparison before coding — this framing alone often earns strong signal.

SECTION 17 — Multiple Language Templates

Java

```java
public int missingNumber(int[] nums) {
    int i = 0, n = nums.length;
    while (i < n) {
        int correct = nums[i];
        if (correct < n && nums[i] != nums[correct]) {
            int tmp = nums[i]; nums[i] = nums[correct]; nums[correct] = tmp;
        } else i++;
    }
    for (i = 0; i < n; i++) if (nums[i] != i) return i;
    return n;
}
```

JavaScript

```javascript
function missingNumber(nums) {
    let i = 0, n = nums.length;
    while (i < n) {
        const correct = nums[i];
        if (correct < n && nums[i] !== nums[correct]) {
            [nums[i], nums[correct]] = [nums[correct], nums[i]];
        } else i++;
    }
    for (i = 0; i < n; i++) if (nums[i] !== i) return i;
    return n;
}
```

PHP

```php
function missingNumber(array $nums): int {
    $i = 0; $n = count($nums);
    while ($i < $n) {
        $correct = $nums[$i];
        if ($correct < $n && $nums[$i] !== $nums[$correct]) {
            [$nums[$i], $nums[$correct]] = [$nums[$correct], $nums[$i]];
        } else $i++;
    }
    for ($i = 0; $i < $n; $i++) if ($nums[$i] !== $i) return $i;
    return $n;
}
```

Python

```python
def missing_number(nums):
    i, n = 0, len(nums)
    while i < n:
        correct = nums[i]
        if correct < n and nums[i] != nums[correct]:
            nums[i], nums[correct] = nums[correct], nums[i]
        else:
            i += 1
```

```
    for i in range(n):
        if nums[i] != i:
            return i
    return n
```

Go

```
func missingNumber(nums []int) int {
    i, n := 0, len(nums)
    for i < n {
        correct := nums[i]
        if correct < n && nums[i] != nums[correct] {
            nums[i], nums[correct] = nums[correct], nums[i]
        } else {
            i++
        }
    }
    for i = 0; i < n; i++ {
        if nums[i] != i {
            return i
        }
    }
    return n
}
```

C++

```
int missingNumber(vector<int>& nums) {
    int i = 0, n = nums.size();
    while (i < n) {
        int correct = nums[i];
        if (correct < n && nums[i] != nums[correct]) swap(nums[i], nums[correct]);
        else i++;
    }
    for (i = 0; i < n; i++) if (nums[i] != i) return i;
    return n;
}
```

SECTION 18 — Dry Runs

Small Input

`nums = [3, 0, 1]` (Missing Number, values 0..n)

```
i=0: nums[0]=3, but 3 == n(3), out of swap range → i++
i=1: nums[1]=0, correct index 0. nums[0]=3   0 → swap → [0, 3, 1]
i=1: nums[1]=3, out of range (3==n) → i++
i=2: nums[2]=1, correct index 1. nums[1]=3   1 → swap → [0, 1, 3]
i=2: nums[2]=3, out of range → i++
Loop ends. Verify: nums[0]=0  , nums[1]=1  , nums[2]=3 2 → missing = 2
```

Large Input (Conceptual)

For an array of 10^6 elements with values 1..10^6 (one missing), the placement loop performs at most 10^6 total swaps (each swap permanently places one element), followed by a single O(10^6) verification scan — total O(2×10^6), confirming linear time regardless of scale.

Corner Case

`nums = [1,1]` (Set Mismatch-style duplicate): `i=0`: `nums[0]=1`, correct index 0, `nums[0]==nums[0]` already → advance. `i=1`: `nums[1]=1`, correct index 0, `nums[1]=1 == nums[0]=1` → duplicate detected, advance without swapping (avoiding infinite loop) → final scan reveals index 1 has wrong value, identifying the duplicate/missing pair correctly.

SECTION 19 — Advanced Concepts

- **Cyclic Sort for minimum swaps to sort:** the number of swaps Cyclic Sort performs to fully sort a permutation equals `n - (number of cycles in the permutation's cycle decomposition)` — a beautiful connection to group theory (permutation cycle decomposition) that occasionally surfaces in "minimum swaps to sort" interview variants.
- **Radix Sort / Bucket Sort as generalizations:** when values aren't bounded to exactly `[1,n]` but are bounded integers in a known range (or have a fixed number of digits), Radix Sort (digit-by-digit counting sort passes) achieves O(d·(n+k)) time, another non-comparison-based approach worth knowing as a sibling technique.
- **Dutch National Flag as a 3-value special case:** partitioning `{0,1,2}` in one pass with three pointers (`low`, `mid`, `high`) is a direct relative of Cyclic Sort's "value tells you where to go" philosophy, specialized to exactly three buckets.
- **Interview hack:** whenever you see "array of size n, values in range [1,n] or [0,n-1]" verbatim, treat it as an almost-certain Cyclic Sort signal — this exact phrasing is a deliberate, recognizable setup used across dozens of problems.

SECTION 20 — Senior Engineer Insights

Staff engineers recognize that Cyclic Sort's real lesson is broader than the specific algorithm: **whenever your data has extra structural information beyond "comparable," exploit it to break the general O(n log n) comparison-sort lower bound.** This same principle underlies radix sort, counting sort, and bucket sort, and shows up in production systems as ID-slot allocation, bitmap-based set representations, and direct-addressing schemes. Interviewers evaluate whether a candidate can spot this "special structure" opportunity — the bounded value range — rather than defaulting to a generic `sort()` call or hashmap, which is the more common but asymptotically inferior choice.

SECTION 21 — Cheat Sheet

```
PATTERN: Cyclic Sort / Sorting Techniques
RECOGNIZE: "array of size n, values in range 1..n or 0..n-1" + missing/duplicate detection + O(1) space
TEMPLATE:
    i = 0
    while i < n:
        correct = nums[i] (adjusted for 0/1-indexing)
        if in-range and nums[i] != nums[correct]: swap
        else: i++
```

```
    final scan: nums[i] != expected(i) reveals anomalies
COMPLEXITY: O(n) time, O(1) space
KEY PROOF: each swap permanently places at least one element home; total swaps bounded by n
WATCH FOR: out-of-range guarding, infinite loops from wrong swap condition, stability requirements
DOESN'T APPLY WHEN: value range doesn't match array size, need stable/non-mutating sort
```

SECTION 22 — Revision Notes (5-Minute Review)

- Cyclic Sort applies when array size n matches the value range (1..n or 0..n-1).
- Swap each value to its "home" index (value - 1 or value); advance only when already correct or duplicate detected.
- Final verification scan (`arr[i] != i+1`) reveals missing/duplicate values.
- O(n) time, O(1) space — beats sorting (O(n log n)) and hashing (O(n) space) for this narrow case.
- Guard against out-of-range values explicitly (First Missing Positive variant).
- General sorting (Merge/Quick/Counting/Radix/Bucket) remains essential for broader, unstructured sorting needs.

SECTION 23 — Practice Roadmap

Stage	Focus	Recommended LeetCode Problems
Beginner	Basic cyclic sort mechanics	Missing Number (268), Set Mismatch (645)
Intermediate	Multiple anomaly detection	Find All Numbers Disappeared in an Array (448), Find All Duplicates in an Array (442)
Advanced	Guarded/negative range handling	First Missing Positive (41), Find the Duplicate Number (287)
Expert	General sorting mastery + advanced applications	Sort Colors (75), Sort an Array (912, implement Merge/Quick Sort), Maximum Gap (164)

SECTION 24 — Memory Tricks

- **Mnemonic:** "**V**alue **I**s **T**he **I**ndex" (VITI) — the value itself tells you exactly where it belongs.
- **Visualization:** A **hotel guest walking directly to their badge-numbered room** — no comparisons needed, just direct placement.
- **Recognition shortcut:** "n numbers, range 1 to n" (or 0 to n-1) + missing/duplicate → Cyclic Sort, immediately.

SECTION 25 — Final Summary

Cyclic Sort achieves O(n) time and O(1) space by exploiting a narrow but powerful structural fact: **when array values are bounded to exactly match the array's index range, each value**

directly encodes its own correct position, eliminating the need for comparison-based sorting entirely. The single most important thing to remember forever: **the moment you see "array of size n with values from 1 to n (or 0 to n-1)" combined with a missing/duplicate detection ask, this is Cyclic Sort, not a general sort or a hashmap** — and the broader lesson, applicable across Radix Sort, Counting Sort, and Bucket Sort too, is to always ask "does my data have extra structure beyond comparability that I can exploit to beat the O(n log n) comparison-sort barrier?"